Adquisición de Datos de Corriente y Potencia Alterna (AC) Método no Invasivo

Ingeniería Mecánica y Electromecánica

Poma Carita Gary Javier
poma.9163997@gmail.com

Docente: Ing. Vidher W. Bustillos D.

Resumen

El informe describe la implementación de un sistema de adquisición de datos de corriente alterna y potencia eléctrica mediante un método no invasivo utilizando un microcontrolador Arduino y un transformador de corriente (CT). El objetivo principal fue medir la corriente sin contacto directo con el conductor, procesar la señal mediante un circuito de acondicionamiento, digitalizarla y almacenarla en un archivo CSV para su análisis en Python. Los resultados obtenidos muestran un comportamiento estable de las variables medidas, permitiendo además la visualización en tiempo real mediante gráficas. Se concluye que el sistema implementado es eficiente para monitoreo eléctrico sin alterar el sistema físico.

Palabras clave: adquisición de datos, corriente alterna, transformador de corriente, Arduino, potencia, método no invasivo.

Current Data Acquisition Using a Hall Effect Sensor

Abstract

This paper presents the design and implementation of a data acquisition system for alternating current and electrical power using a non-invasive method based on an Arduino microcontroller and a current transformer (CT). The objective was to measure current without direct electrical contact, process the signal, digitize it, and store it in a CSV file for analysis in Python. The results showed stable measurements and real-time visualization through graphs. The developed system proved to be efficient for electrical monitoring without affecting the physical system.

Keywords: data acquisition, alternating current, current transformer, Arduino, power, monitoring.

Introducción

La adquisición de datos es una herramienta fundamental en la ingeniería moderna, ya que permite analizar sistemas físicos en tiempo real sin interferir en su funcionamiento. En sistemas eléctricos, la medición de corriente y potencia es esencial para evaluar el consumo energético y el desempeño de equipos.

El uso de transformadores de corriente (CT) permite realizar mediciones de manera no invasiva, evitando la interrupción del circuito eléctrico. Estos dispositivos convierten la corriente en una señal proporcional que puede ser interpretada por un microcontrolador.

ADQUISICIÓN DE DATOS DE CORRIENTE Y POTENCIA

El objetivo de la práctica fue implementar un sistema capaz de medir corriente alterna, calcular potencia y energía, almacenar los datos en formato CSV y visualizarlos en tiempo real mediante Python.

Desarrollo de la Investigación

Solución propuesta

Se implementó un sistema basado en:

- Sensor de corriente tipo CT (transformador de corriente SCT-013 30A/1V)
- Microcontrolador Arduino
- Circuito de acondicionamiento de señal
- Software en Python para visualización y almacenamiento

El circuito de Acondicionamiento de señal para el CT, consiste en un divisor de tensión que desplaza la señal alterna sinusoidal 2.5V junto con un capacitor que filtra la señal y finalmente el amplificador operacional está configurado en modo seguidor para estabilizar la señal.

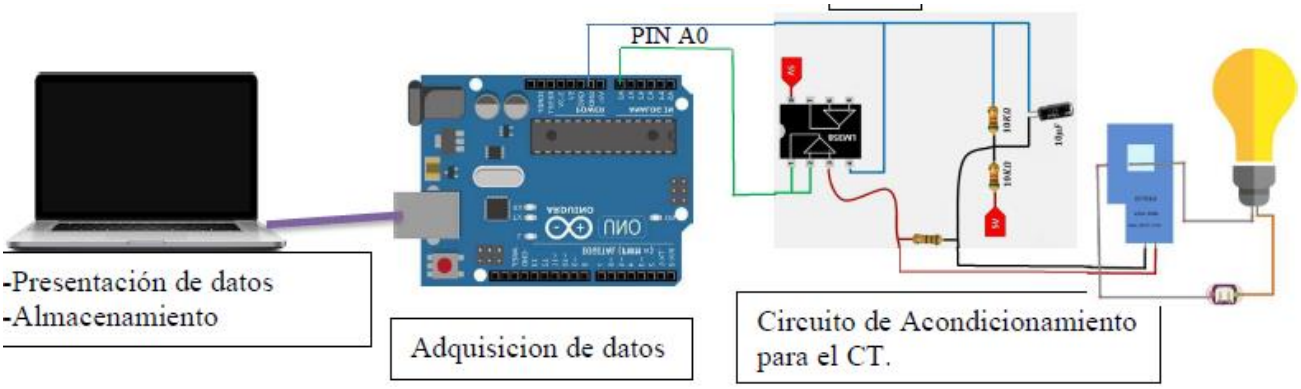
El sistema mide la corriente RMS, calcula la potencia eléctrica y la energía consumida mediante integración en el tiempo.

Requerimientos materiales y equipos

- Arduino UNO
- Transformador de corriente (SCT-013 30A/1V)
- Resistencias 10K para acondicionamiento
- Protoboard
- Cables de conexión
- Computadora
- Software Arduino IDE
- Python (librerías: PySerial, Matplotlib, NumPy)

Diagrama en bloques

Figura 1. *Diagrama en bloques.*



Fuente: elaboración propia.

Codificación de los programas

IDE Arduino:

Figura 2. *Codificación en el IDE Arduino.*

```
#include <Wire.h>
// SENSOR
const int pinSensor = A0;

// ADC
const float Vref = 5.0;
const int resolucionADC = 1023;

// SCT-013-030
const float factorConversion = 22.5; // A/V

// Voltaje red
const float VoltajeRed = 230.0;

// Muestreo
const int tiempoMuestreo = 220;

// Energía
float energia = 0;
unsigned long tiempoAnterior = 0;

void setup() {
  Serial.begin(9600);
  tiempoAnterior = millis();
}

void loop() {

  unsigned long inicio = millis();

  float sumaCuadrados = 0;
  int contador = 0;
  float offset = 0;

  // OFFSET
  while (millis() - inicio <
tiempoMuestreo) {
    offset += analogRead(pinSensor);
    contador++;
  }
  offset /= contador;
  offset = (offset * Vref) / resolucionADC;

  // RMS
  inicio = millis();
  contador = 0;
  sumaCuadrados = 0;
```

```
  while (millis() - inicio <
tiempoMuestreo) {

    int lectura = analogRead(pinSensor);
    float voltaje = (lectura * Vref) /
resolucionADC;
    float senal = voltaje - offset;
    sumaCuadrados += senal * senal;
    contador++;
  }

  float Vrms = sqrt(sumaCuadrados /
contador);

  // CORRIENTE
  float Irms = Vrms * factorConversion;
  float Irms_mA = Irms * 1000;

  // POTENCIA
  float Potencia = VoltajeRed * Irms*0.6;

  // ENERGÍA (Wh)
  unsigned long tiempoActual = millis();
  float dt = (tiempoActual -
tiempoAnterior) / 1000.0; // segundos
  tiempoAnterior = tiempoActual;

  energia += (Potencia * dt) / 3600.0;

  // FILTRO
  static float I_filtrado = 0;
  static float P_filtrado = 0;

  float alpha = 0.2;

  I_filtrado = alpha * Irms_mA + (1 -
alpha) * I_filtrado;
  P_filtrado = alpha * Potencia + (1 -
alpha) * P_filtrado;

  // SERIAL
  Serial.print("Corriente RMS:");
  Serial.print(I_filtrado, 2);
  Serial.print(" mA | Potencia:");
  Serial.print(P_filtrado, 2);
  Serial.println(" W");

  delay(500);
}
```

Fuente: elaboración propia.

Codificación en Python:

Figura 3. *Codificación en Python.*

```python
import serial
import matplotlib.pyplot as plt
import matplotlib.animation as animation
from matplotlib.widgets import Button
import csv
import time
import numpy as np
import atexit

PUERTO = 'COM4'
BAUDRATE = 9600

arduino = serial.Serial(PUERTO, BAUDRATE, timeout=1)
time.sleep(2)

#CSV
archivo = open("datos.csv", "w", newline="",
encoding="utf-8")
csvw = csv.writer(archivo, delimiter=';')
csvw.writerow(["Tiempo (s)", "Corriente (mA)", "Potencia
(W)", "Energía (Wh)"])

def cerrar_archivo():
    archivo.close()
    arduino.close()
atexit.register(cerrar_archivo)
# VARIABLES
t = []
I = []
P = []
E = []

inicio = time.time()
t_prev = inicio
energia = 0
# FIGURA
fig, ax = plt.subplots()
plt.subplots_adjust(bottom=0.25)

ax2 = ax.twinx()
ax.set_title("Adquisicion de datos de Corriente Metodo no
Invasivo")
lineI, = ax.plot([],[],'b-',label="Corriente (mA)")
lineP, = ax2.plot([],[],'r--',label="Potencia (W)")
lines = [lineI, lineP]
labels = [l.get_label() for l in lines]
ax.legend(lines, labels)

texto = ax.text(0.02,0.95,'',transform=ax.transAxes,
               verticalalignment='top')
ax.set_xlabel("Tiempo (s)")
ax.set_ylabel("Corriente (mA)")
ax2.set_ylabel("Potencia (W)")
# BOTON
pausa = False
def toggle(event):
    global pausa
    pausa = not pausa

btn_ax = plt.axes([0.8,0.05,0.1,0.075])
btn = Button(btn_ax,"Pausar")
btn.on_clicked(toggle)
```

```python
def update(frame):
    global t_prev, energia
    if pausa:
        return lineI, lineP, texto
    try:
        d =
arduino.readline().decode(errors='ignore').strip()
        if "Corriente RMS:" in d and "Potencia:" in d:
            partes = d.split("|")
            i =
float(partes[0].split(":")[1].replace("mA",""))
            p =
float(partes[1].split(":")[1].replace("W",""))

            ahora = time.time()
            dt = ahora - t_prev
            t_prev = ahora
            # energía Wh
            energia += (p * dt) / 3600
            tiempo = ahora - inicio
            t.append(tiempo)
            I.append(i)
            P.append(p)
            E.append(energia)

            if len(t) > 100:
                t.pop(0); I.pop(0); P.pop(0); E.pop(0)
            # GUARDADO CSV
            csvw.writerow([
                f"{tiempo:.2f}",
                f"{i:.2f}",
                f"{p:.2f}",
                f"{energia:.4f}"
            ])
            archivo.flush()

            lineI.set_data(t,I)
            lineP.set_data(t,P)

            # ESCALA DINÁMICA
            if I:
                ax.set_ylim(0, max(I)*1.2 + 10)
            if P:
                ax2.set_ylim(0, max(P)*1.2 + 10)

            ax.set_xlim(max(0, t[-1]-10), t[-1])
            # texto
            texto.set_text(
                f"I: {i:.2f} mA\n"
                f"P: {p:.2f} W\n"
                f"E: {energia:.4f} Wh"
            )
    except Exception as e:
        print(e)

    return lineI, lineP, texto
#
ani = animation.FuncAnimation(fig, update, interval=100,
blit=False)

plt.show()
```

Fuente: elaboración propia.

ADQUISICIÓN DE DATOS DE CORRIENTE Y POTENCIA

El programa permite:

- Leer datos seriales desde Arduino
- Extraer corriente y potencia
- Calcular energía
- Guardar datos en CSV
- Graficar en tiempo real

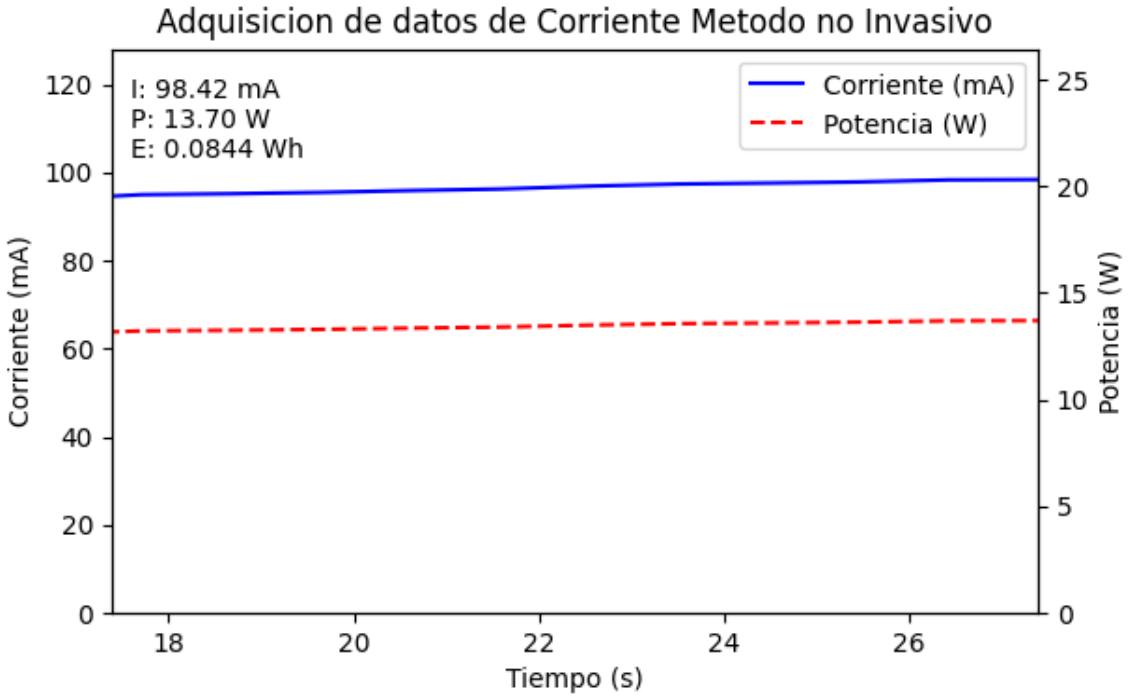
Resultados Obtenidos

Los datos adquiridos fueron almacenados en un archivo *.CSV que contiene tres columnas: tiempo (s), corriente (mA), Potencia (W) y Energía (Wh).

Se observa que la corriente presenta variaciones en el tiempo, con valores aproximados entre **97 mA y 98 mA**, lo cual coincide con la corriente medida con el amperímetro.

La potencia presenta comportamiento estable (~13.5 W).
La energía aumenta progresivamente con el tiempo.

Figura 4. *Grafica de las Variaciones de Corriente y Potencia en el Foco de 13W obtenida con Python*



Fuente: elaboración propia.

Tabla 1. *Datos de tiempo, corriente, potencia y energía obtenidos del archivo CSV.*

Tiempo (s)	***Corriente (mA)***	***Potencia (W)***	***Energia (Wh)***
22.55	96.93	13.49	0.0661
23.51	97.41	13.56	0.0697
24.48	97.63	13.59	0.0734
25.45	97.88	13.63	0.0771
26.42	98.35	13.69	0.0807
27.39	98.42	13.70	0.0844
28.36	97.73	13.60	0.0881
29.33	97.68	13.60	0.0917
30.29	97.80	13.61	0.0954
31.26	97.62	13.59	0.0991
32.23	96.88	13.49	0.1027
33.20	96.86	13.48	0.1063
34.17	96.78	13.47	0.1099

Fuente: elaboración propia.

Discusión y conclusiones

El sistema desarrollado cumple con el objetivo de medir corriente y potencia mediante un método no invasivo. La implementación del transformador de corriente permitió realizar mediciones sin alterar el circuito eléctrico, lo cual es una ventaja importante en aplicaciones reales.

El uso de Python permitió visualizar los datos en tiempo real y almacenarlos correctamente, facilitando su análisis posterior.

Entre las principales conclusiones:

- El método no invasivo es seguro y eficiente
- El sistema presenta buena estabilidad en las mediciones
- La integración hardware-software es adecuada para monitoreo energético si además se implementa otro sistema para medir Voltaje.

Como limitación, se puede mencionar la dependencia de la calibración del sensor CT para mejorar la precisión.

Referencias bibliográficas

Bustillos D., V. W. (2026). *POO_ADQUISICION_1* [Presentación de PowerPoint]. Facultad de Ingeniería, Universidad Mayor de San Andrés.

midudev. (2025, marzo 25). *Curso COMPLETO de Python DESDE CERO para Principiantes 2025* [Video]. YouTube. https://youtu.be/TkN2i-_4N4g

ANEXOS

Figura 5. *Hoja de datos.*

UNIVERSIDAD MAYOR DE SAN ANDRÉS
FACULTAD DE INGENIERÍA

LMEC-237 LABORATORIO DE MÉTODOS NUMÉRICOS Y DISEÑO DE ALGORITMOS III
Ing. Vidher W. Bustillos D.

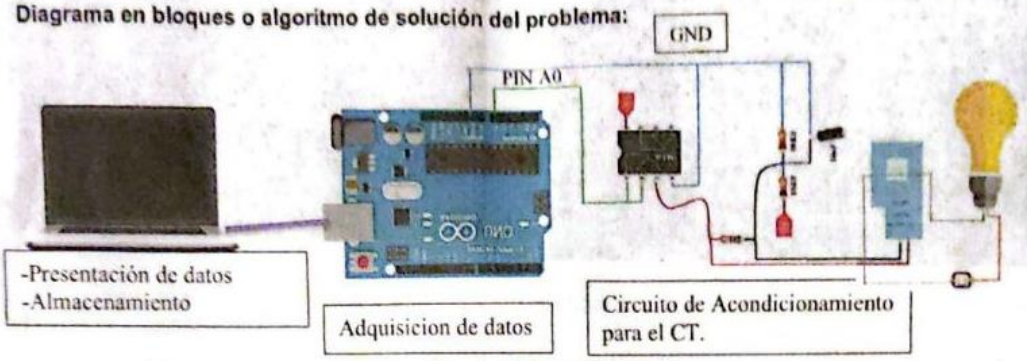
HOJA DE DATOS

LABORATORIO No: 4	FECHA:09/04/2026	Vo.Bo. Ing. Vidher W. Bustillos D.					
Adquisición de Datos de Corriente y Potencia Alterna (AC) Método no invasivo							
PARTICIPANTES (Apellidos Nombres)		Llenado datos (15)	Presentación de resultados (10)	Funcionamiento Adecuado (30)	Informe en Plataforma (15)	INFORME Físico (30)	NOTA LABORATORIO (100)
1.-Poma Canta Gary Javier		15	10	30			
2.-							

Nombre competo de la Práctica a desarrollar:
Adquisición de Datos de Corriente Alterna y Potencia, Método no Invasivo con Microcontrolador y Transformador de Corriente (CT)

Antecedentes del problema a solucionar:
Adquirir datos de un sistema físico real con el método no invasivo, con un microcontrolador, graficar y almacenar los datos adquiridos en Python.

Diagrama en bloques o algoritmo de solución del problema:



Fuente: elaboración propia.

Codificación completa del programa:

```
import serial
import matplotlib.pyplot as plt
import matplotlib.animation as animation
from matplotlib.widgets import Button
import csv
import time
import numpy as np
import atexit

PUERTO = "COM4"
BAUDRATE = 9600

arduino = serial.Serial(PUERTO, BAUDRATE, timeout=1)
time.sleep(2)

#CSV
archivo = open("datos.csv", "w", newline="",
encoding="utf-8")
csvw = csv.writer(archivo, delimiter=';')
csvw.writerow(["Tiempo (s)", "Corriente (mA)", "Potencia
(W)", "energia (Wh)"])

def cerrar_archivo():
    archivo.close()
    arduino.close()

atexit.register(cerrar_archivo)

# VARIABLES
t = []
I = []
P = []
```

```
E = []

inicio = time.time()
t_prev = inicio
energia = 0

# FIGURA
fig, ax = plt.subplots()
plt.subplots_adjust(bottom=0.25)

ax2 = ax.twinx()
ax.set_title("Adquisicion de datos de Corriente Metodo no
Invasivo")
lineI, = ax.plot([],[],'b-',label="Corriente (mA)")
lineP, = ax2.plot([],[],"r--",label="Potencia (W)")

lines = [lineI, lineP]
labels = [l.get_label() for l in lines]
ax.legend(lines, labels)

texto = ax.text(0.02,0.95,"",transform=ax.transAxes,
                verticalalignment='top')

ax.set_xlabel("Tiempo (s)")
ax.set_ylabel("Corriente (mA)")
ax2.set_ylabel("Potencia (W)")

# BOTON
pausa = False

def toggle(event):
    global pausa
```

Fuente: elaboración propia.

Figura 6. *Hoja de datos.*

UNIVERSIDAD MAYOR DE SAN ANDRÉS
FACULTAD DE INGENIERÍA

LMEC-237 LABORATORIO DE MÉTODOS NUMÉRICOS Y DISEÑO DE ALGORITMOS III
Ing. Vidher W. Bustillos D.

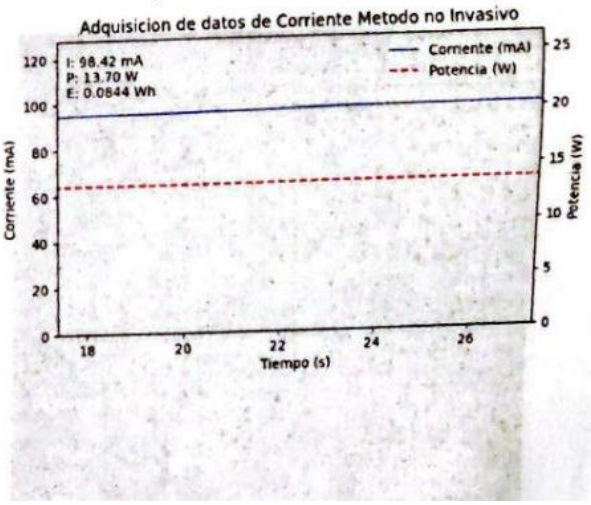
```
    pausa = not pausa

btn_ax = plt.axes([0.8,0.05,0.1,0.075])
btn = Button(btn_ax,"Pausar")
btn.on_clicked(toggle)

def update(frame):
    global t_prev, energia

    if pausa:
        return lineI, lineP, texto

    try:
        d = arduino.readline().decode(errors='ignore').strip()

        if "Corriente RMS:" in d and "Potencia:" in d:
            partes = d.split("|")

            i = float(partes[0].split(":")[1].replace("mA",""))
            p = float(partes[1].split(":")[1].replace("W",""))

            ahora = time.time()
            dt = ahora - t_prev
            t_prev = ahora

            # energía Wh
            energia += (p * dt) / 3600

            tiempo = ahora - inicio

            t.append(tiempo)
            I.append(i)
            P.append(p)
            E.append(energia)

            if len(t) > 100:
                t.pop(0); I.pop(0); P.pop(0); E.pop(0)

            # GUARDADO CSV
            csvw.writerow([
                f"{tiempo:.2f}",        # 2 decimales
                f"{i:.2f}",             # 2 decimales
                f"{p:.2f}",             # 2 decimales
                f"{energia:.4f}"        # 4 decimales
            ])
            archivo.flush()

            lineI.set_data(t,I)
            lineP.set_data(t,P)

            # ESCALA DINÁMICA
            if I:
                ax.set_ylim(0, max(I)*1.2 + 10)
            if P:
                ax2.set_ylim(0, max(P)*1.2 + 10)

            ax.set_xlim(max(0, t[-1]-10), t[-1])

            # texto
            texto.set_text(
                f"I: {i:.2f} mA\n"
                f"P: {p:.2f} W\n"
                f"E: {energia:.4f} Wh"
            )
        else:
            pass
    except Exception as e:
        print(e)

    return lineI, lineP, texto

# RUN
ani = animation.FuncAnimation(fig, update, interval=100,
blit=False)

plt.show()
```

Captura de ventanas del funcionamiento adecuado del algoritmo o codificación del programa

Archivo *csv:

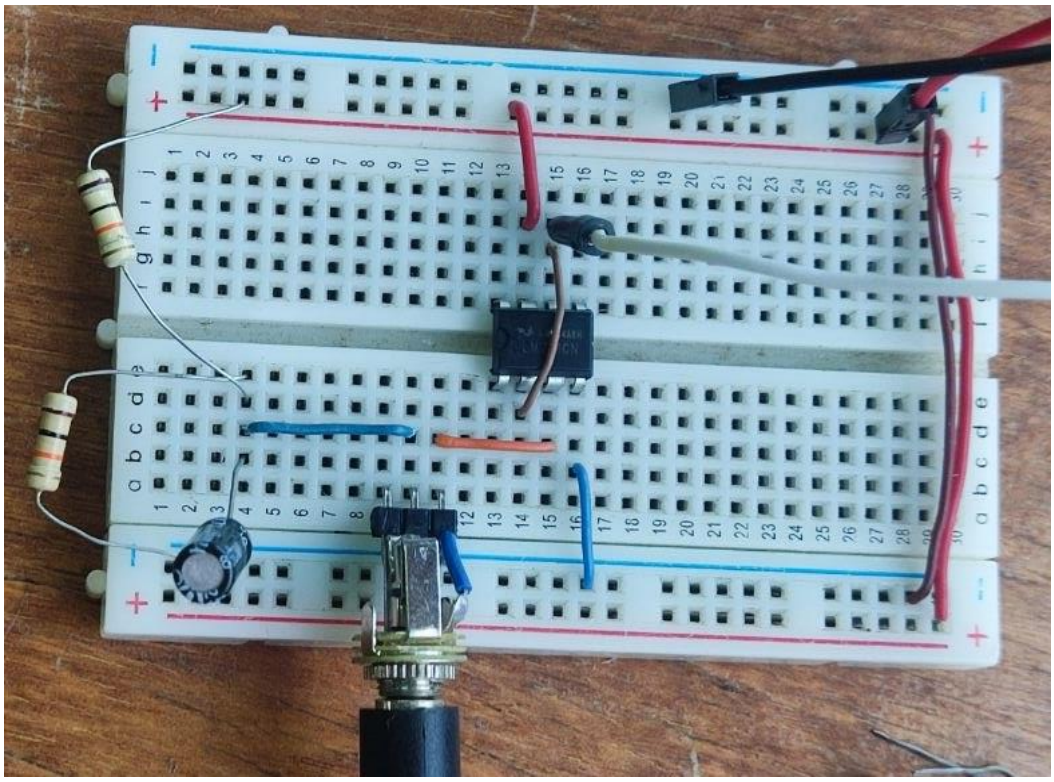
Tiempo (s)	Corriente (mA)	Potencia (W)	Energia (Wh)
22.55	96.93	13.49	0.0661
23.51	97.41	13.56	0.0697
24.48	97.63	13.59	0.0734
25.45	97.88	13.63	0.0771
26.42	98.35	13.69	0.0807
27.39	98.42	13.70	0.0844
28.36	97.73	13.60	0.0881
29.33	97.68	13.60	0.0917
30.29	97.80	13.61	0.0954
31.26	97.62	13.59	0.0991
32.23	96.88	13.49	0.1027
33.20	96.86	13.48	0.1063
34.17	96.78	13.47	0.1099

Grafica:



Fuente: elaboración propia.

Figura 7. *Circuito de acondicionamiento para el SCT-013.*



Fuente: elaboración propia.